QUESTION 1

Use class, objects, constructors while coding Bank Account Management System You are tasked with creating a simple bank account management system in Python. Implement a class called BankAccount with the following specifications: The class should have private instance variables for account number, account holder name, and balance. Include a constructor to initialize these variables. Implement getter and setter methods for each instance variable to ensure encapsulation. Implement methods to deposit and withdraw money from the account. Ensure that the withdraw method checks if the account has sufficient balance before allowing withdrawal. Write a Python program to demonstrate the functionality of the BankAccount class by creating instances, depositing and withdrawing money, and displaying account information.

```
class BankAccount:
    def __init__(self, acc_no, name, balance):
        self.__acc_no = acc_no
        self.__name = name
        self.__balance = balance
    def get_acc_no(self):
        return self.__acc_no
    def get_name(self):
        return self.__name
    def get_balance(self):
        return self.__balance
    def set_name(self, name):
        self.__name = name
    def set_balance(self, balance):
        if balance > 0:
            self.__balance = balance
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print("Amount Deposited: ", amount)
        else:
            print("Invalid deposit amount ")
    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            print("Amount withdrawn: ", amount)
        else:
            print("Insufficient Balance")
    def display(self):
        print("\n--- Account Details---")
        print("Account No: ", self.__acc_no)
        print("Name: ", self.__name)
        print("Balance: ", self.__balance)
```

```
from bank_accounts import BankAccount
acc1 = BankAccount(101, "Pratiksha", 5000)
acc1.display()
acc1.deposit(2000)
acc1.withdraw(3000)
acc1.withdraw(10000)
acc1.display()
```

OUTPUT

```
from bank_accounts import BankAccount
```

```
acc1 = BankAccount(101, "Pratiksha", 5000)
acc1.display()
acc1.deposit(2000)
acc1.withdraw(3000)
acc1.withdraw(10000)
acc1.display()
```

QESTION 2

Employee Management System You are developing an employee management system for a company. Implement a class called Employee with the following specifications: The class should have private instance variables for employee ID, name, and salary. Include a constructor to initialize these variables. Implement getter and setter methods for each instance variable. Implement a method to display employee information. Implement a method to give a salary hike to an employee. The method should accept a percentage value by which the salary will be increased. Write a Python program to demonstrate the functionality of the Employee class by creating instances, displaying employee information, giving salary hikes, and displaying updated employee information.

```
class Employee:

    def __init__(self, emp_id, name, salary):
        self.__emp_id = emp_id
        self.__name = name
        self.__salary = salary
    def get_emp_id(self):
        return self.__emp_id
    def get_name(self):
        return self.__name
    def get_salary(self):
        return self.__salary
    def set_name(self, name):
        self.__name = name
    def set_salary(self, salary):
        if salary >= 0:
            self.__salary = salary
        else:
            print("Invalid Salary.")
    def display(self):
        print("\n--- Employee Details ---")
        print("ID: ", self.__emp_id)
        print("Name: ", self.__name)
        print("Salary: ", self.__salary)
    def give_hike(self, percent):
        if percent > 0:
            hike = self.__salary * percent / 100
            self.__salary += hike
            print(f"Salary increased by {percent}%")
        else:
            print("Invalid percentage.")
```

```python
from employeeassign import Employee

emp1 = Employee(101, "Pratiksha", 30000)
emp1.display()
emp1.give_hike(10)
emp1.display()
```

OUTPUT

```
--- Employee Details ---

ID:  101

Name:  Pratiksha

Salary:  30000

Salary increased by 10%



--- Employee Details ---

ID:  101

Name:  Pratiksha

Salary:  33000.0
```